

RFM Analysis Report

Introduction

This phase of the project focuses on performing RFM (Recency, Frequency, Monetary) analysis using Python on the cleaned transactional dataset obtained from the Basic Data Cleaning phase. RFM analysis is a widely used customer segmentation technique that evaluates customer purchasing behavior based on how recently they purchased, how frequently they purchased, and how much money they spent.

The final outcome of this phase is an RFM-based Excel sheet, where each row represents a single customer, enriched with RFM metrics and RFM scores that can be directly used for customer segmentation and business decision-making.

Input Dataset

The input dataset is the cleaned Excel file exported at the end of the Basic Data Cleaning phase.

This dataset contains only valid sales transactions and includes the following important columns:

- **Customer ID**
- **Invoice**
- **Invoice Date**
- **Quantity**
- **Price**
- **Total Amount**

Each row represents a transaction-level record, and multiple rows may belong to the same customer.

Objective of RFM Analysis

The objective of this phase is to:

1. Convert transaction-level data into customer-level data
2. Calculate Recency, Frequency, and Monetary values
3. Assign RFM scores using quantile-based logic
4. Export the final RFM dataset as an Excel file

Procedure

Tools Used

- **Python (VScode)**
- **Pandas** for data manipulation and aggregation
- **NumPy** for numerical support

Code Logic Explanation:

Step 1: Import Required Libraries

```
import numpy as np
import pandas as pd
```

- **pandas** is used to load the Excel file and perform grouping, aggregation, and column operations.
- **numpy** provides numerical support and is commonly used alongside Pandas for data analysis.
- Importing these libraries prepares the Python environment for RFM analysis.

Step 2: Load the Cleaned Excel Dataset

```
df = pd.read_excel("Phase_2.xlsx")
```

- `pd.read_excel()` reads the cleaned Excel file into a Pandas DataFrame.
- The DataFrame `df` now contains the cleaned transaction-level data.
- This dataset will be used as the input for RFM calculations.

Step 3: Inspect the Dataset: `df.head()`

- Displays the first five rows of the dataset.
- Helps verify column names, data structure, and overall correctness before analysis.

Step 4: Convert Invoice Date to Datetime Format

```
df['InvoiceDate (R)'] = pd.to_datetime(df['InvoiceDate (R)'])
```

- Converts the InvoiceDate column from text/object format to datetime format.
- Datetime conversion is required to perform date-based calculations for Recency.
- Without this step, Python cannot compute date differences.

Step 5: Define the Recency Anchor Date

```
analysis_date = df['InvoiceDate (R)'].max() + pd.Timedelta(days=1)
```

- `df['InvoiceDate (R)'].max()` finds the most recent transaction date in the dataset.
- `pd.Timedelta(days=1)` adds one day to that date.
- This new date is called the **recency anchor date**.
- Recency is calculated by comparing each customer's last purchase date with this anchor.

Step 6: Aggregate Data at Customer Level

```
rfm = df.groupby('Customer ID').agg({  
    'InvoiceDate (R)': lambda x: (analysis_date - x.max()).days,  
    'Invoice (F)': 'nunique',  
    'TotalAmount (M)': 'sum'  
})
```

- `groupby('Customer ID')` groups all transactions belonging to the same customer.
- This converts transaction-level data into customer-level data.

Inside `agg()`:

- **Recency:**
 - `x.max()` gets the customer's last purchase date.
 - `(analysis_date - x.max()).days` calculates the number of days since last purchase.
- **Frequency:**
 - `'nunique'` counts the number of unique invoices per customer.
- **Monetary:**
 - `'sum'` calculates the total amount spent by the customer.

Each customer now has exactly **one row**.

Step 7: Rename Aggregated Columns

```
rfm.columns = ['Recency', 'Frequency', 'Monetary']
```

- Renames the aggregated columns for clarity.
- Makes the dataset easier to understand and aligns with standard RFM terminology.

Step 8: Reset Index

```
rfm = rfm.reset_index()
```

- Converts Customer ID from an index back into a normal column.
- This makes the dataset easier to export and analyze further.

Step 9: RFM Base Table

At this stage, the RFM base table contains:

	A	B	C	D
1	Customer ID	Recency	Frequency	Monetary

Each row represents **one customer**, not one transaction.

Step 10: Recency Scoring

```
rfm['R_Score'] = pd.qcut(
    rfm['Recency'],
    5,
    labels=[5, 4, 3, 2, 1]
)
```

- pd.qcut() divides customers into 5 equal-sized groups (quintiles).
- Lower recency values indicate more recent purchases.
- Therefore, the scoring is reversed:
 - Most recent customers → score 5
 - Least recent customers → score 1

Step 11: Frequency Scoring

```
rfm['F_Score'] = pd.qcut(  
    rfm['Frequency'].rank(method='first'),  
    5,  
    labels=[1, 2, 3, 4, 5]  
)
```

- Higher frequency indicates more loyal customers.
- rank(method='first') resolves ties to avoid binning errors.
- Customers are scored from 1 (lowest frequency) to 5 (highest frequency).

Step 12: Monetary Scoring

```
rfm['M_Score'] = pd.qcut(  
    rfm['Monetary'],  
    5,  
    labels=[1, 2, 3, 4, 5]  
)
```

- Customers are divided into five monetary groups.
- Higher spending customers receive higher scores.
- This captures customer value from a revenue perspective.

Step 13: Convert Scores to Numeric Format

```
rfm[['R_Score', 'F_Score', 'M_Score']] = rfm[['R_Score', 'F_Score', 'M_Score']].astype(int)
```

- Converts score columns from categorical to integer type.
- Numeric scores are easier to analyze, compare, and segment.

Step 14: Create RFM Code

```
rfm['RFM_Code'] = (  
    rfm['R_Score'].astype(str) +  
    rfm['F_Score'].astype(str) +  
    rfm['M_Score'].astype(str)  
)
```

- Combines individual R, F, and M scores into a single three-digit RFM code.
- Example:
 - R = 5, F = 4, M = 5 → RFM Code = 545
- This compact code represents the overall customer value.

Step 15: Export RFM Dataset to Excel

```
rfm.to_excel("RFM_Output.xlsx", index=False)
```

- Exports the final RFM dataset to an Excel file.
- index=False prevents the DataFrame index from being saved as a column.
- This Excel file is the final output of the RFM analysis phase.

Final RFM Output

The exported Excel file contains:

- Customer ID
- Recency
- Frequency
- Monetary
- R Score
- F Score
- M Score
- RFM Code

Each row represents a **single customer**, making the dataset suitable for segmentation and reporting.

Summary of RFM Analysis Phase

1. Cleaned transactional data was loaded into Python.
2. Invoice dates were converted to datetime format.
3. A recency anchor date was defined.
4. Data was aggregated at customer level to compute RFM metrics.
5. Quantile-based scoring was applied to standardize RFM values.
6. RFM codes were generated to represent customer value.
7. The final RFM dataset was exported as an Excel file.

This phase successfully converts cleaned transactional data into a structured, customer-level RFM dataset, enabling advanced customer segmentation and business insights.